Phase-Adaptive Quantization: Design-Space Exploration of 4-Bit MMA Rescale Pipelines

Yichong Zhang
yichongz@mit.edu

Wenye Xiong
wenyexio@mit.edu

***Abstract*—Existing 4-bit MMA formats bundle a single rescale pipeline and apply it uniformly to compute-bound prefill and bandwidth-bound decode, even though system-level work has shown the two phases warrant separate hardware treatment. For a custom inference accelerator the rescale pipeline is itself a design variable that should be co-designed with the workload—and, when it pays, with the inference phase. We reformulate the W4A4 datapath as a structured search space, evaluate it across LLM, VLM, and VLA representative GEMMs in both phases (prefill and decode), and report three findings with direct implications for next-generation accelerator design. (i) The industry-standard NVFP4-like reference is Pareto-dominated in every one of the six (workload, phase) cells: custom silicon can replace its FP32 rescale stages with cheaper operators at <0.005 cosine drop, saving 48–60% energy end-to-end—the gap lives in the rescale stages and the memory traffic they induce, not in the FP4 MAC itself. (ii) Phase-aware datapaths are not a universal win: extending prefill/decode disaggregation down to the rescale pipeline buys 8.2% for VLA, 0.6% for LLM, and 0% for VLM, so the second silicon mode should be budgeted only for certain workloads. (iii) We propose an efficient two-step search procedure for this design space—an accuracy-floor pass that admits a block-granularity family, followed by a cost-driven pick within the family—which on all six (workload, phase) cells returns the same configuration as exhaustive sweep. An analytic per-output cost model further shows the headline savings only grow as the input length M scales to production prompt lengths.**

I. Introduction

Modern AI inference splits into two regimes with opposite hardware profiles. **Prefill** ($M\gg1$) is compute-bound: the energy floor sits in the multiply-accumulate datapath. **Decode** ($M=1$) is memory-bound: the energy floor sits in streaming weights from HBM. System-level work has already exploited this asymmetry by physically disaggregating the two phases onto separate GPU pools—DistServe [16] reports 7.4× more served requests under SLO, and Splitwise [17] shows that pairing H100s for prefill with A100s for decode is TCO-optimal. Yet at the *datapath* level every deployed 4-bit MMA format [1], [2], [4], [8]–[13] still bundles a single rescale pipeline—fixed number of levels, fixed block size, fixed scale and accumulator precision—and applies it uniformly to both phases. The bundling is conservative for good reason: GPU vendors must be safe across arbitrary models and tasks, including training and inference.

Custom AI inference accelerators do not face that constraint. Once the model family, prefill/decode mix, and accuracy budget are known, the rescale pipeline becomes a hardware-design variable—and an expensive one. On Llama-style FFN GEMMs, our breakdown (Table V) shows that the rescale stages and the GLB/DRAM traffic they induce dominate NVFP4 W4A4 energy, while the FP4 MAC datapath itself contributes a single-digit-percent share, so even a constant-factor reduction in the rescale path changes the system-level energy story. Yet no prior format-level work treats the rescale pipeline itself as a per-phase design variable.

This paper asks what rescale pipeline a custom W4A4 accelerator should build, and how strongly the answer depends on the inference phase. We treat the pipeline as a structured design space rather than a fixed format, formalize it through a small canonical operator algebra, and evaluate it across representative LLM, VLM, and VLA workloads in both phases under a fixed accuracy floor. Three findings emerge. The industry-standard NVFP4-like reference is Pareto-dominated in every cell we evaluate. Phase-aware datapaths are materially beneficial only for VLA-class workloads, marginal for LLMs, and provably zero-gain for VLM—a workload-conditional verdict the format literature does not provide. For this design space we propose a two-step search procedure (accuracy-floor admission, then within-family cost ordering) that on all six cells recovers the exhaustive-sweep best, providing a candidate alternative to full DSE for future workloads. Together these reframe W4A4 quantization research at the datapath level: the right output is a *design-time methodology*, not yet another single 4-bit format.

Contributions: (i) A structured search space for the W4A4 rescale pipeline, decomposed across three orthogonal axes, that subsumes MXFP4 and NVFP4 as named points (§III). (ii) A canonical Einsum formalization that turns every configuration into a subset of seven operators, sharing one mapper across the entire space and attributing energy operator-by-operator (§III-B). (iii) A two-step efficient search procedure for this design space (accuracy-floor admission $\rightarrow$ within-family cost pick) that on each of our six cells recovers the same configuration as exhaustive sweep at $\mathcal{O}(1)$ accuracy probes per cell (§IV-B). (iv) An analytic per-output cost decomposition showing the headline savings strengthen monotonically with prompt length (§IV-D).

II. Related Work

Prior 4-bit quantization splits along three axes; we position our contribution against each in turn.

The first line proposes *new format-level pipelines* as single hand-designed points in the same space we expose. The OCP MX specification [2] standardised MXFP4/6/8 with E8M0 block scales (evaluated empirically in [3]); NVFP4 [1] adds an FP8-per-block + FP32-per-tensor two-level scheme on Blackwell tensor cores; HiFloat4 [8], M^2XFP [12], and MX+ [13] extend the family with hierarchical scaling, metadata augmentation, or rescaled exponents; and [11] maps the block-size/scale-format frontier under direct-cast PTQ. Each fixes a single rescale topology and applies it to all phases of inference; none asks whether prefill and decode warrant different points in the space.

A second line modifies the *quantization algorithm* on top of a fixed format-level pipeline—GPTQ [4] via Hessian-based reconstruction, AWQ [5] via salient-weight preservation, SmoothQuant [6] via activation-to-weight outlier migration. These methods are orthogonal to the datapath choice we sweep and stack with any configuration in our space.

A third line co-designs adjacent *hardware axes*. HAQ [7] learns per-layer mixed-precision via RL on top of off-the-shelf datapaths; AXE [9] treats accumulator precision as tunable at the algorithm level; MixPE [10] performs Pareto DSE over dequantization *placement* without varying the rescale topology. Tool-wise our methodology drives Timeloop+Accelergy [14], [15] through AccelForge—the same mapper/energy stack used in prior accelerator DSE—but applies it to a new variable: the rescale pipeline itself.

III. Methodology

A. Search-space decomposition

We fix the weight format to E2M1 and decompose the rescale pipeline into three orthogonal axes that span existing 4-bit proposals as named points. *Rescale topology* is the number of scaling levels: zero (raw FP4 multiply), one (a per-block scale shared by b contiguous elements, as in MXFP4), or two (a per-block fine scale plus a per-tensor coarse scale, as in NVFP4). *Block granularity* b is the number of elements covered by one fine scale, with $b \in \{16, 32\}$ covering the production points without combinatorial blow-up. *Scale and accumulator precision* controls per-level scale storage (E8M0 power-of-two shift; FP16; FP32) and the MAC accumulator (FP32 or aggressive FP16). Pruning combinations that are invalid (accumulator narrower than operand) or strictly dominated (e.g. FP32 fine inside a 1-level pipeline whose 2-level FP32 sibling exists) leaves the 10 representative configurations C0–C9 in Table I.

B. The rescale pipeline as canonical Einsum operators

To make every configuration a single change to a shared mapping problem, we express the entire W4A4 pipeline as a composition of seven canonical Einsum operators. Let $A[m,k] \in \mathbb{R}^{M\times K}$, $W[k,n] \in \mathbb{R}^{K\times N}$, fine block size b, optional coarse (tensor) scales T_A, T_W:

$$\text{(1) BlockScaleA: } s_A[m, \tfrac{k}{b}] = \frac{\max_{k'} |A[m, k{:}k{+}b]|}{r_{\max}}, \tag{1}$$

$$\text{(2) BlockQuantA: } A_q[m,k] = \mathrm{round}\big(A[m,k] / s_A[m, \tfrac{k}{b}]\big), \tag{2}$$

$$\text{(3) MatMulBlock: } Y_{\mathrm{acc}}[m,n] = \textstyle\sum_k A_q[m,k]\, W_q[k,n], \tag{3}$$

$$\text{(4) RescaleBlockA: } Y' = Y_{\mathrm{acc}} \cdot s_A, \tag{4}$$

$$\text{(5) RescaleBlockW: } Y'' = Y' \cdot s_W, \tag{5}$$

$$\text{(6) RescaleTensorA: } Y''' = Y'' \cdot T_A, \tag{6}$$

$$\text{(7) RescaleTensorW: } Y = Y''' \cdot T_W. \tag{7}$$

Operator (3) becomes `MatMulNVFP4` when the accumulation is sliced over coarse blocks (i.e. in 2-level pipelines). W is analogously block-quantized offline; weight-side operators (s_W, T_W) reuse the same kernels. Each configuration is the subset of $\{1, \ldots, 7\}$ activated by its *topology*, plus a choice of scale precision per level and accumulator precision (Table II). This is exactly the operator list emitted by AccelForge for each run, so the breakdown `energy_per_einsum` ties back to a specific row of Table II with no glue logic.

TABLE I
The 10 configurations spanning the W4A4 rescale pipeline search space. C1 reproduces MXFP4; C7 reproduces NVFP4. Acc. is the MAC accumulator format.

ID	Topology	b	Fine sc.	Coarse sc.	Acc.	Reference
C0	0-level	–	–	–	FP32	ideal-W4A4
C1	1-level	32	E8M0	–	FP32	MXFP4-like
C2	1-level	16	E8M0	–	FP32	
C3	1-level	16	FP16	–	FP32	
C4	1-level	16	FP32	–	FP32	
C5	2-level	16	E8M0	FP16	FP32	
C6	2-level	16	FP16	FP16	FP32	
C7	2-level	16	FP32	FP32	FP32	NVFP4-like
C8	1-level	16	FP16	–	**FP16**	
C9	2-level	16	E8M0	FP16	**FP16**	

TABLE II
Configuration → active Einsum subset. "+t" marks operators (6)/(7) that appear only in 2-level configurations.

ID	(1)	(2)	(3)	(4)	(5)	(6)	(7)
C0			✓				
C1, C2, C3, C4, C8	✓	✓	✓	✓	✓		
C5, C6, C7, C9	✓	✓	✓/NVFP4	✓	✓	+t	+t

C. Hardware and accuracy evaluation

Hardware model: We drive Timeloop [14] + Accelergy [15] through AccelForge on a parameterized 8×8 PE-array accelerator at 45 nm: 64 KB shared GLB, 128 B per-PE RF, a dedicated FP4 MAC unit, and configurable RescaleFineMAC / RescaleCoarseMAC units whose component table is re-generated per configuration from the active Einsum subset. Mappings are picked by Timeloop's standard search; the same

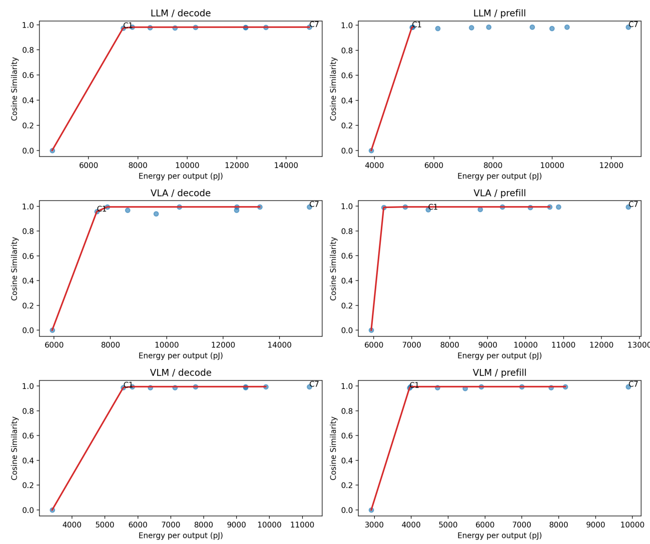

Fig. 1. Accuracy–energy Pareto panels for the 3 workloads $\times$ 2 phases sweep. C7 (NVFP4-like) is consistently high-energy; C1 (MXFP4-like) sits near the low-energy knee for LLM prefill and both VLM phases; C8/C9 win the b=16-required cells.

per-config mapping is reused across all (M, N, K) triples in a workload to keep configurations comparable.

Workloads: We pick one representative GEMM per application class: LLM Llama-2 FFN $(M, 11008, 4096)$, VLM Qwen2-VL ViT $(M, 3072, 3072)$, and VLA OpenVLA action head $(M, 256, 4096)$. Each is evaluated in prefill (M=128) and decode (M=1). Sec. IV-D shows analytically that the choice of prefill M does not change the conclusions for any larger M.

Accuracy: We use per-layer cosine similarity between the quantized output and an FP16 reference on deterministic tensor snapshots, with eval slices m_{eval}=4 for prefill and 1 for decode and n_{eval}=16 across all configs. The admissibility floor is $\cos \geq 0.98$, a level that prior microscaling work [3], [11] has correlated with downstream task quality; we revisit this proxy in Sec. V. The full sweep is $10{\times}3{\times}2 = 60$ hardware runs and 60 matched accuracy runs.

IV. Results

A. Claim 1: NVFP4 is Pareto-dominated in every cell

Table III reports the lowest-energy configuration that clears the 0.98 cosine floor in each cell. Across all six (workload, phase) pairs the C7 NVFP4-like reference is strictly dominated, by 48–60% in energy at <0.005 cosine drop. Fig. 1 shows the underlying Pareto panels: C7 sits at consistently high energy, while C1, C8, and C9 collectively cover the low-energy knee.

B. Claim 2: A two-step efficient search procedure for this space

Rather than re-running a 10-config exhaustive sweep for every new workload, we propose the following *search procedure* that exploits the structure of our search space:

TABLE III
Best configuration per (workload, phase) under cosine ≥ 0.98. The 6 cells select 3 distinct configurations; C7 is never selected.

Workload	Phase	Best	pJ/out	Cosine	vs. C7
LLM	Prefill	C1	5262	0.9843	−58.1%
LLM	Decode	C8	7767	0.9807	−48.0%
VLM	Prefill	C1	3952	0.9836	−60.0%
VLM	Decode	C1	5557	0.9874	−50.4%
VLA	Prefill	C9	6263	0.9895	−50.7%
VLA	Decode	C8	7891	0.9950	−47.6%

TABLE IV
Floor-pass matrix. "✓" = cosine ≥ 0.98; "×" = fails. Bold cells are admitted. The single member of the b=32 family is C1; for the b=16 family we report C8 as a representative low-cost member (other b=16 points behave identically except VLA decode where C9 also fails at 0.968).

Workload	Phase	C1 (b=32)	C8 (b=16)	Admitted
LLM	Prefill	✓ (.984)	✓ (.985)	b=32
LLM	Decode	× (.974)	✓ (.981)	b=16
VLM	Prefill	✓ (.984)	✓ (.994)	b=32
VLM	Decode	✓ (.987)	✓ (.994)	b=32
VLA	Prefill	× (.971)	✓ (.993)	b=16
VLA	Decode	× (.958)	✓ (.995)	b=16

1) *Floor pass.* Compute cosine for the cheapest member of each block-granularity family (b=32 vs. b=16). Admit the coarsest family that clears the accuracy floor (here 0.98).
2) *Within-family cost pick.* Inside the admitted family, select the configuration whose active Einsum subset has the lowest total energy at the cell's (M, N, K), evaluated by the analytic per-output decomposition of Sec. IV-D.

We *evaluate the procedure* by checking, on each of our six (workload, phase) cells, whether it returns the same configuration as the exhaustive sweep of Table III. Table IV reports the Step 1 outcome (admitted family) for every cell, and the result of Step 2 matches Table III in $6/6$ cases. This is an *evaluation on six cells*, not a generalisation guarantee: whether the procedure recovers the optimum on workloads outside our three classes, on different accuracy floors, or on enriched search spaces with mixed scale-format axes is a question we return to in Sec. V. The remainder of this subsection walks through the three non-trivial cells and the dominated NVFP4 reference, showing how operator-cost algebra and floor outcomes determine each procedure output.

VLM collapses to C1. Both VLM phases pass the b=32 floor (Table IV); the family has a single member, so Step 2 is trivial. One MXFP4-like mode suffices and a second phase-specific datapath would be wasted area.

LLM and VLA decode pick C8. At M=1, weight-side fine ops are unamortised and C9's extra RescaleTensor stages add $\Theta(N)$ per-output cost without payback; VLA decode further drops C9 below the floor. C8—1-level FP16 fine with FP16 accumulator—is the cheapest b=16 survivor.

TABLE V

PER-COMPONENT ENERGY BREAKDOWN FOR LLM PREFILL (M=128, 1.41×10^6 OUTPUT ELEMENTS). ENERGIES ARE ABSOLUTE TOTALS IN GIGA-PJ. THE MAC DATAPATH IS IDENTICAL; THE 2.39× GAP IS RESCALE AND RESCALE-INDUCED MEMORY.

Component	C1 (MXFP4)	C7 (NVFP4)	C7/C1
FP4 MAC	1.154	1.154	1.0×
RescaleFine MAC	0.011	2.669	247×
RescaleCoarse MAC	0.000	1.340	—
GLB	1.808	3.615	2.0×
DRAM	3.719	8.214	2.2×
RF	0.721	0.721	1.0×
Total	**7.41**	**17.71**	**2.39×**

VLA prefill flips to C9. At M=128, fine RescaleA ops scale as $\Theta(MK/b)$ and the FP16→E8M0 swap saves ∼33× per fine op, outweighing the new RescaleTensor cost; C9 cosine clears the floor. This is the only cell where the best switches across phases by operator algebra rather than floor outcome.

NVFP4 (C7) is dominated by construction. C7 uses FP32 at both rescale levels, the most expensive realisation of any operator in Eqs. (4)–(7); every floor-passing 2-level alternative (C5/C6/C9) replaces FP32 with FP16 or E8M0 at <0.005 cosine cost. Table V confirms the mechanism on LLM prefill: the FP4 MAC datapath is identical, so the entire 2.39× gap lives in rescale stages and the memory traffic they induce.

C. Claim 3: Phase-aware datapaths are conditional

To quantify the system-level value of phase-aware silicon we combine the per-phase bests with a deployment-weighted metric $E(\alpha) = \alpha E_{\text{prefill}} + (1-\alpha)E_{\text{decode}}$ and compare three strategies: fixed NVFP4 (C7 in both phases), the best single fixed configuration that clears both phase floors, and our phase-adaptive choice. Phase-adaptive selection saves 50–60% versus fixed NVFP4 across $\alpha \in \{0.2, 0.3, 0.5, 0.8\}$. Versus the best fixed configuration the gain becomes workload-conditional: $\approx 0\%$ for VLM, 0.6% for LLM, and 8.2% for VLA at α=0.5 (Table VI, Fig. 2). Phase adaptation pays only when prefill and decode select different best configs. This happens in two distinct ways: the cosine floor admits different families (LLM, 0.6%), or the same family's cheapest algebra differs across M (VLA, 8.2%, where C9 wins prefill and C8 wins decode within the same b=16 family). VLM falls into neither case and gains 0%. Phase adaptation is therefore a workload-conditional design choice, not a universal upgrade.

D. Claim 4: M-scaling only strengthens the conclusions

A natural concern is that our M=128 profiling point is unrepresentative of production prompt lengths ($M \sim 10^3$–10^4). We show analytically that all three preceding claims hold or strengthen as M grows.

V. FUTURE WORK

(i) Held-out validation: evaluate the procedure on new model classes (Mistral, Stable Diffusion, Diffusion Policy) and broader cosine floors to obtain a generalisation rate. *(ii) End-to-end task metrics:* replace the cosine proxy with perplexity / task accuracy / episode success on the selected configurations. *(iii) Re-targeting and composition:* the Einsum formalisation is template-agnostic; composing our datapath with weight-only methods such as AWQ [5] could relax the floor and shift the phase-adaptive verdict.

VI. CONCLUSION

The W4A4 rescale pipeline should be a design-space variable for custom accelerators. Our efficient search procedure matches exhaustive sweep within the space, and phase-aware silicon pays only on certain workloads. After all, we are coming up with a new methodology, not another fixed format.

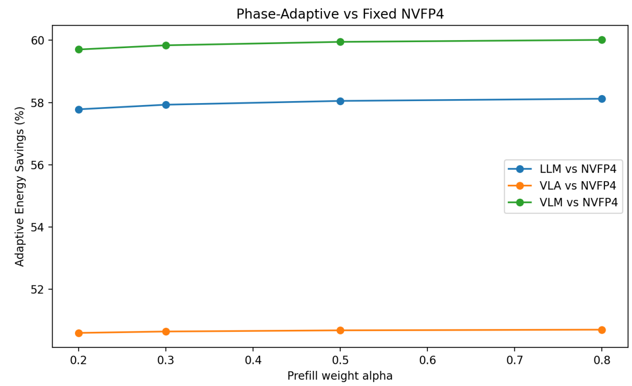

Fig. 2. Phase-adaptive energy savings vs. fixed-NVFP4 (C7) and vs. best single fixed configuration as α varies. The vs.-best-fixed gap is workload-dependent: 0% for VLM, 0.6% for LLM, 8.2% for VLA.

TABLE VI

WEIGHTED ENERGY SAVINGS AT $\alpha = 0.5$. PHASE ADAPTATION HELPS ONLY WHEN PREFILL AND DECODE SELECT DIFFERENT BEST CONFIGS—EITHER VIA DIFFERENT ADMITTED FAMILIES (LLM) OR VIA THE SAME FAMILY'S CHEAPEST ALGEBRA DIFFERING ACROSS M (VLA).

Workload	Adaptive	Best-fixed	vs. NVFP4	vs. best-fixed
LLM	(C1, C8)	C8	−58.1%	−0.6%
VLM	(C1, C1)	C1	−59.9%	0.0%
VLA	(C9, C8)	C8	−50.7%	−8.2%

[Placeholder] M-scaling figure
x: $M \in \{1, 16, 64, 128, 512, 2048, 8192\}$;
y: phase-adaptive saving vs. C7 (%).
Three curves: LLM, VLM, VLA prefill.
Expected: monotone non-decreasing per analytic model.

Fig. 3. Phase-adaptive energy saving vs. C7 as M grows; M=128 is a conservative lower bound for Table III.

References

[1] NVIDIA, "Introducing NVFP4 for Efficient and Accurate Low-Precision Inference," NVIDIA Technical Blog, 2025; B. Castro et al., "Pretraining Large Language Models with NVFP4," arXiv:2509.25149, 2025.

[2] Open Compute Project, "OCP Microscaling Formats (MX) Specification v1.0," Sept. 2023.

[3] B. Rouhani et al., "Microscaling Data Formats for Deep Learning," arXiv:2310.10537, 2023.

[4] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers," *ICLR*, 2023.

[5] J. Lin et al., "AWQ: Activation-Aware Weight Quantization for On-Device LLM Compression and Acceleration," *MLSys*, 2024 (Best Paper).

[6] G. Xiao et al., "SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models," *ICML*, 2023.

[7] K. Wang, Z. Liu, Y. Lin, J. Lin, and S. Han, "HAQ: Hardware-Aware Automated Quantization with Mixed Precision," *CVPR*, 2019.

[8] Y. Sun et al., "HiFloat4 Format for Language Model Inference," arXiv:2602.11287, 2026.

[9] A. Saha et al., "AXE: Accumulator-Aware Post-Training Quantization," arXiv:2409.17092, 2024.

[10] Z. Zhang et al., "MixPE: Quantization and Hardware Co-design," arXiv:2411.16158, 2024.

[11] T. Jeong et al., "Is Finer Better? Limits of Microscaling Data Formats," arXiv:2601.19026, 2026.

[12] J. Park et al., "M^{2}XFP: Metadata-Augmented Microscaling," *ASPLOS*, 2026.

[13] S. Kim et al., "MX+: Pushing the Limits of Microscaling Formats," *MICRO*, 2025.

[14] A. Parashar et al., "Timeloop: A Systematic Approach to DNN Accelerator Evaluation," *ISPASS*, 2019.

[15] Y. N. Wu, J. S. Emer, and V. Sze, "Accelergy: An Architecture-Level Energy Estimation Methodology for Accelerator Designs," *ICCAD*, 2019.

[16] Y. Zhong et al., "DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving," *OSDI*, 2024.

[17] P. Patel et al., "Splitwise: Efficient Generative LLM Inference Using Phase Splitting," *ISCA*, 2024.

[18] H. Wang et al., "BitVLA: 1-bit Vision-Language-Action Models for Robotics Manipulation," arXiv:2506.07530, 2025.